

C Programming

5_arrays

Average score?

	score1	score2	score3	
	77	43	100	
	Паша	Маша	Даша	

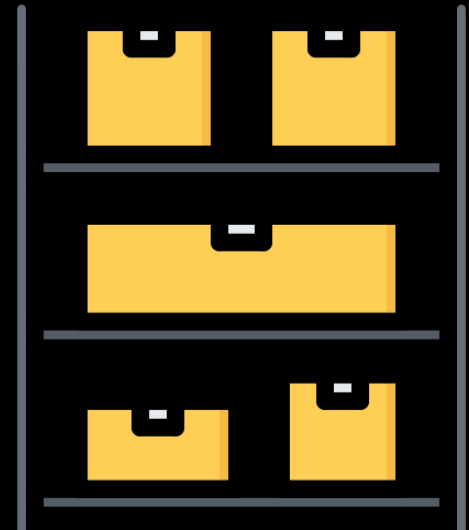
$$\text{avg} = (s1 + s2 + s3) / 3.0$$

Arrays

Массивы используются для хранения нескольких значений в одной переменной вместо объявления отдельных переменных для каждого значения.

Если переменная - это коробка со значением, то массив - это шкаф с этими коробками

Массивы во многих ЯП (в том числе Си) умеют хранить значения **только одного** типа данных, т.е. один массив не может хранить сразу и **целые** и **вещественные** типы данных



Arrays

```
int scores[3] = {77, 43, 100};
```

	[0]	[1]	[2]	
Value	77	43	100	
Address	200	204	208	

Массивы в C/C++, Python, Java, Go и многих других ЯП индексируются с 0
Исключения, где массивы начинаются с индекса 1: MATLAB, lua, R, Julia..

```
printf("Scores: ");  
for (int i = 0; i < 3; i++) {  
    printf("%d, ", scores[i]);  
}  
printf("\n");
```

```
#define N 5
```

```
int a[N];
```

```
int b[5] = {1, 2, 3, 4, 5};
```

```
float c[] = {1.5, 3.2, 2.5};
```

```
int d[6] = {[3] = 9};
```

```
char e[N] = {0};
```

```
int a[5] = {5, 6, 7, 8, 9};  
int b[5] = {1, 2, 3, 4, 5};
```

```
b = a; // Error  
a = {5, 3}; // Error
```

```
a[2] = -17; // Ok  
a[0] = b[4]; // Ok
```

```
a[2.5] = 33; // Error  
b[4000] = 100; // Error, segfault  
a[-1] = 8; // Ok, possible segfault
```

Segmentation fault - ошибка во время выполнения программы, когда программа попыталась обратиться к памяти, к которой доступ не разрешен.

array size

```
int m[] = {1, 2, 3}; // 3 элемента  
int array_size = sizeof(m) / sizeof(m[0]); // 12/4=3
```

delete/insert

В статическом (обычном) массиве размер фиксирован:
сократить или расширить длину массива невозможно

"Удалить" можно сдвигом хвоста влево:

```
for (int i = pos; i < n - 1; i++) arr[i] = arr[i + 1];
```

а актуальную длину держать в отдельной переменной (n--).

```
void print_arr(int n, int array[n]) {  
    for (int i = 0; i < n; i++) {  
        printf("array[%d] = %d\n", i, array[i]);  
    }  
    printf("\n");  
}
```

```
int scores[N] = {77, 43, 100, 55, 11};  
print_arr(N, scores);
```

```
int array[M] = {1, 2, 3};  
print_arr(M, array);
```

Задачи на массивах

```
long long sum = 0;
for (int i = 0; i < n; i++) {
    sum += arr[i];
}
float avg = (float)sum / n;    // не sum / n!

int max = arr[0];              // не 0!
for (int i = 1; i < n; i++) {
    if (arr[i] > max) max = arr[i];
}
```

Задачи на массивах

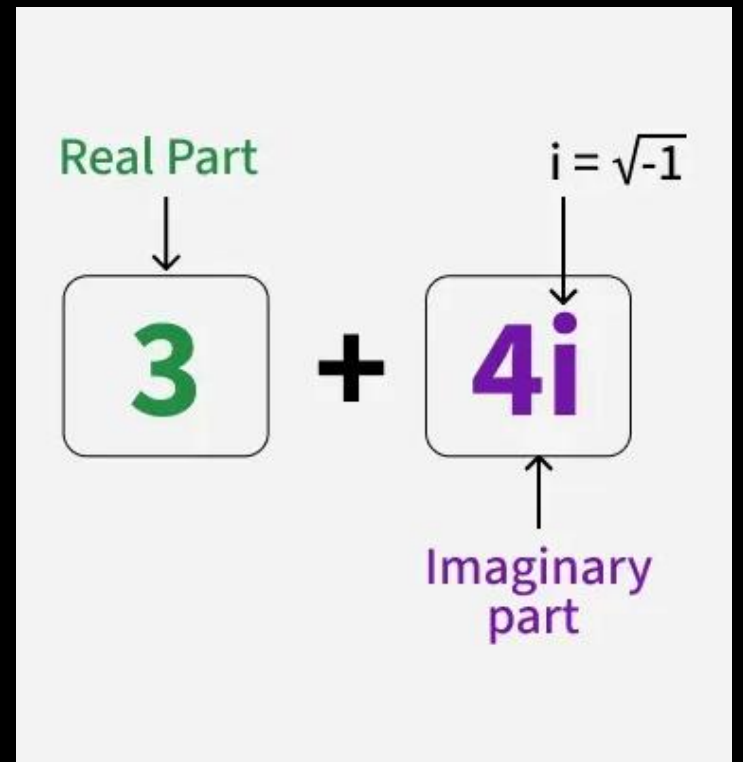
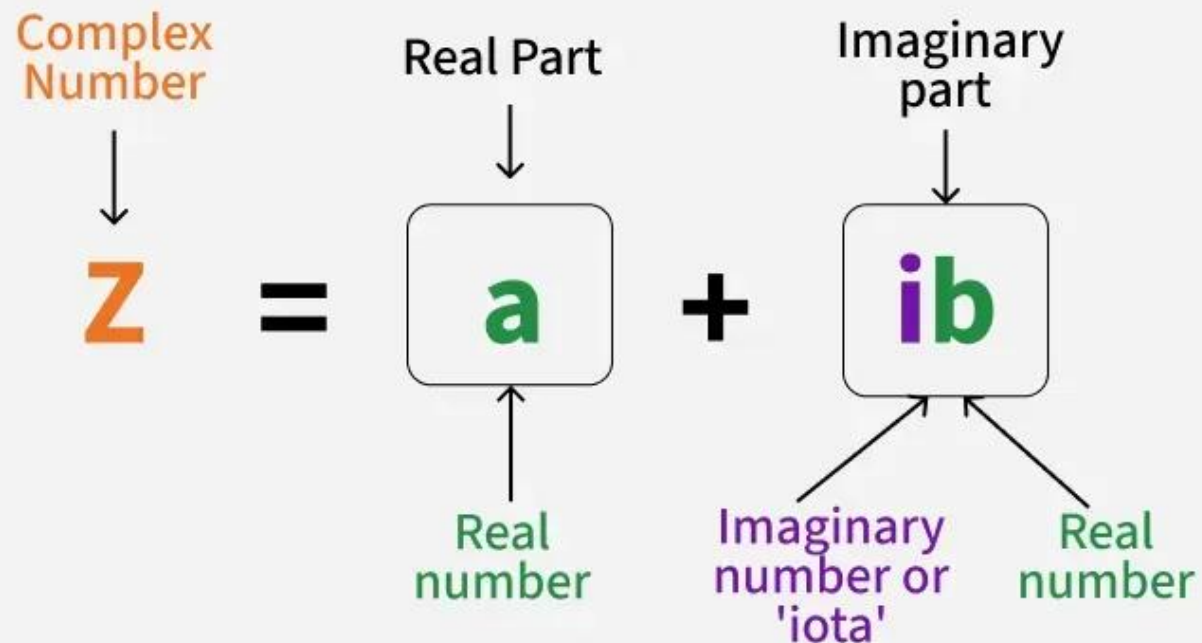
// линейный поиск

```
for (int i = 0; i < n; i++) {  
    if (arr[i] == value) return i;  
}  
return -1;
```

// переворот

```
for (int i = 0; i < n / 2; i++) {  
    int tmp      = arr[i];  
    arr[i]       = arr[n - 1 - i];  
    arr[n-1-i]   = tmp;  
}
```

Массивы в реальном мире



Телефоны "из воздуха" достают большие массивы комплексных чисел, когда работают с Wi-Fi, 4G и 5G.

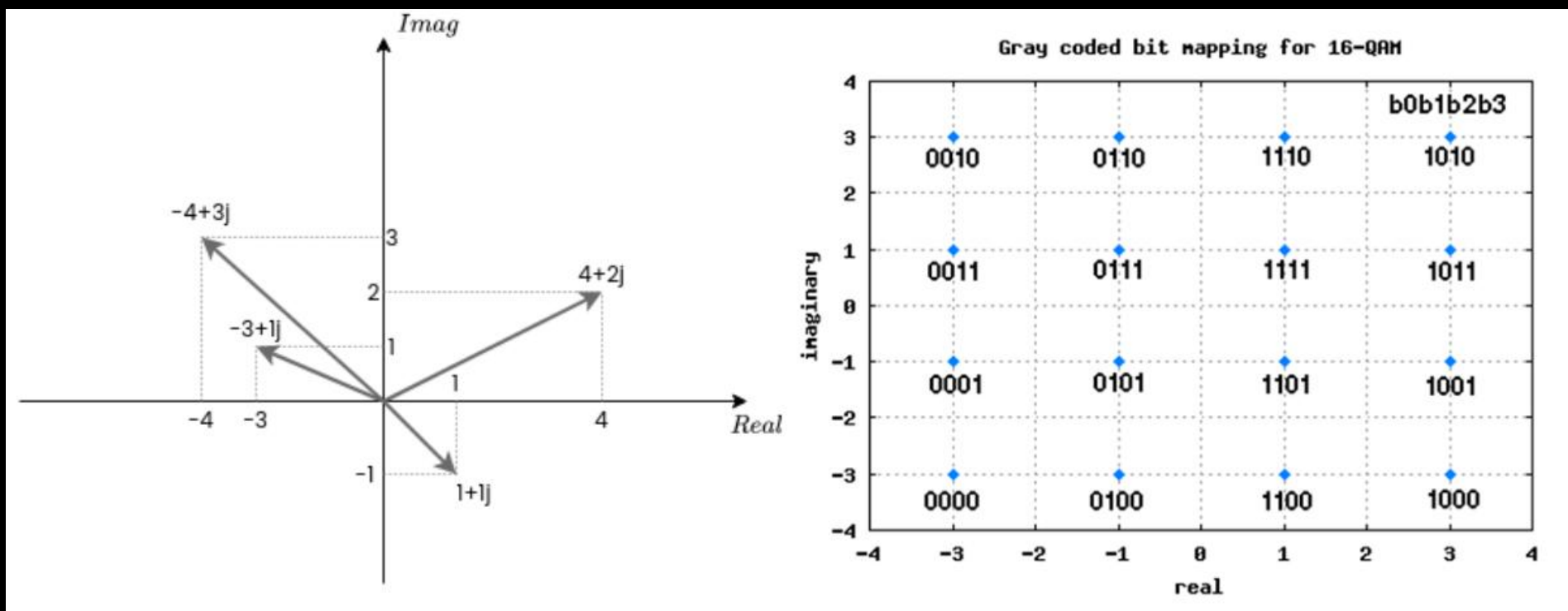
Сигнал в эфире - синусоида. Чтобы вложить в неё данные, меняют два параметра: амплитуду (насколько громко) и фазу (с каким сдвигом).

$$A \times \cos(\omega t + \varphi) = a \times \cos(\omega t) - b \times \sin(\omega t)$$

Пара (a, b) - это комплексное число $a + bi$.

В радио это компоненты I и Q.

Бит в одной точке (а, в) решает модуляция:
BPSK - 1, QPSK - 2, 16-QAM - 4, 64-QAM - 6, 1024-QAM - 10.
Плотнее созвездие - чувствительнее приём:
стало шумно - модем сам падает до 16-QAM.



Реальный приёмник получает IQ-поток:
десятки миллионов значений в секунду и
записывает их массивы.

Допустим нам пришло 10 комплексных чисел,
по 16 бит на часть. Как упаковать их в массив?

```
short iq[20];
```

```
iq[2*i]    // I (реальная часть)
```

```
iq[2*i+1]  // Q (мнимая часть)
```

```
int iq_packed[10];
```

```
(iq_packed[i] & 0xffff)           // I
```

```
((iq_packed[i] >> 16) & 0xffff) // Q
```

Массив чисел = вектор

```
#define DIM 768    // у реальных векторов - сотни-тысячи чисел

// скалярное произведение = "похожесть" двух векторов
int dot(int n, const int a[n], const int b[n]) {
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i] * b[i];
    return sum;
}

// "кот", "кошка", "трактор"... чей вектор ближе к "котёнку"?
// так работает поиск по смыслу, рекомендации и весь ИИ
```

Random numbers

```
#include <stdlib.h>  
// ...  
int random = rand();
```

rand() - ф-ия, что возвращает случайное **целое** число (целое в диапазоне от 0 до RAND_MAX, где RAND_MAX - определен в stdlib.h)

Random numbers

- **srand()** - ф-ия задания seed (семя) для псевдорандома. Seed - некоторое число, что каким-то образом влияет на формулу случайных чисел
- **time()** - ф-ия возвращающая кол-во секунд прошедших с 00:00:00 UTC, 1 Января, 1970 года, называется - Unix timestamp (timestamp - временная метка).

<https://www.unixtimestamp.com/>

```
srand(time(NULL)); // достаточно сделать 1 раз  
printf("Unix timestamp = %ld\n", time(NULL));  
printf("Random [0..99] = %d\n", rand() % 100);
```

Самостоятельно: заполнить массив размером N случайными числами от 0 до 999 и найти в нем минимальный элемент - вывести его на экран

Matrix

Переменная - коробка, массив - шкаф с коробками, матрица - комната со шкафами..

Массивы могут быть не только одномерными, могут быть двумерными.. n -мерными. Двумерные массивы обычно называют словом **матрица**, т.к. они похожи на матрицы (или таблицы) со строками и столбцами. Индексы все также начинаются с нуля!

	Column 0	Column 1	Column 2
Row 0	<code>x[0][0]</code>	<code>x[0][1]</code>	<code>x[0][2]</code>
Row 1	<code>x[1][0]</code>	<code>x[1][1]</code>	<code>x[1][2]</code>
Row 2	<code>x[2][0]</code>	<code>x[2][1]</code>	<code>x[2][2]</code>



```
int matrix[N][N];
for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        matrix[i][j] = i + j + 1;
    }
}

for (int i = 0; i < N; i++) {
    for (int j = 0; j < N; j++) {
        printf("%d ", matrix[i][j]);
    }
    printf("\n");
}
```

Memory

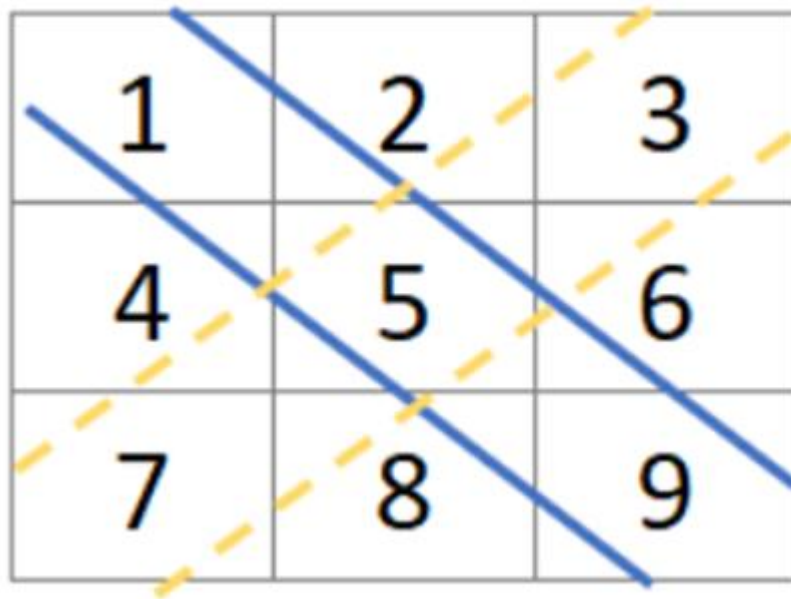
```
int m[3][3] = { {1, 2, 3},  
                {4, 5, 6},  
                {7, 8, 9}};
```

m[0][0] m[0][1] m[0][2] m[1][0] m[1][1] m[1][2] m[2][0] m[2][1] m[2][2]

	1	2	3	4	5	6	7	8	9	
--	----------	----------	----------	----------	----------	----------	----------	----------	----------	--

208 212 216 220 224 228 232 236 240

addr(m[0][2]) = addr(c[0][0]) + 0*3*sizeof(int) + 2*sizeof(int) = 208 + 0 + 2*4 = 216

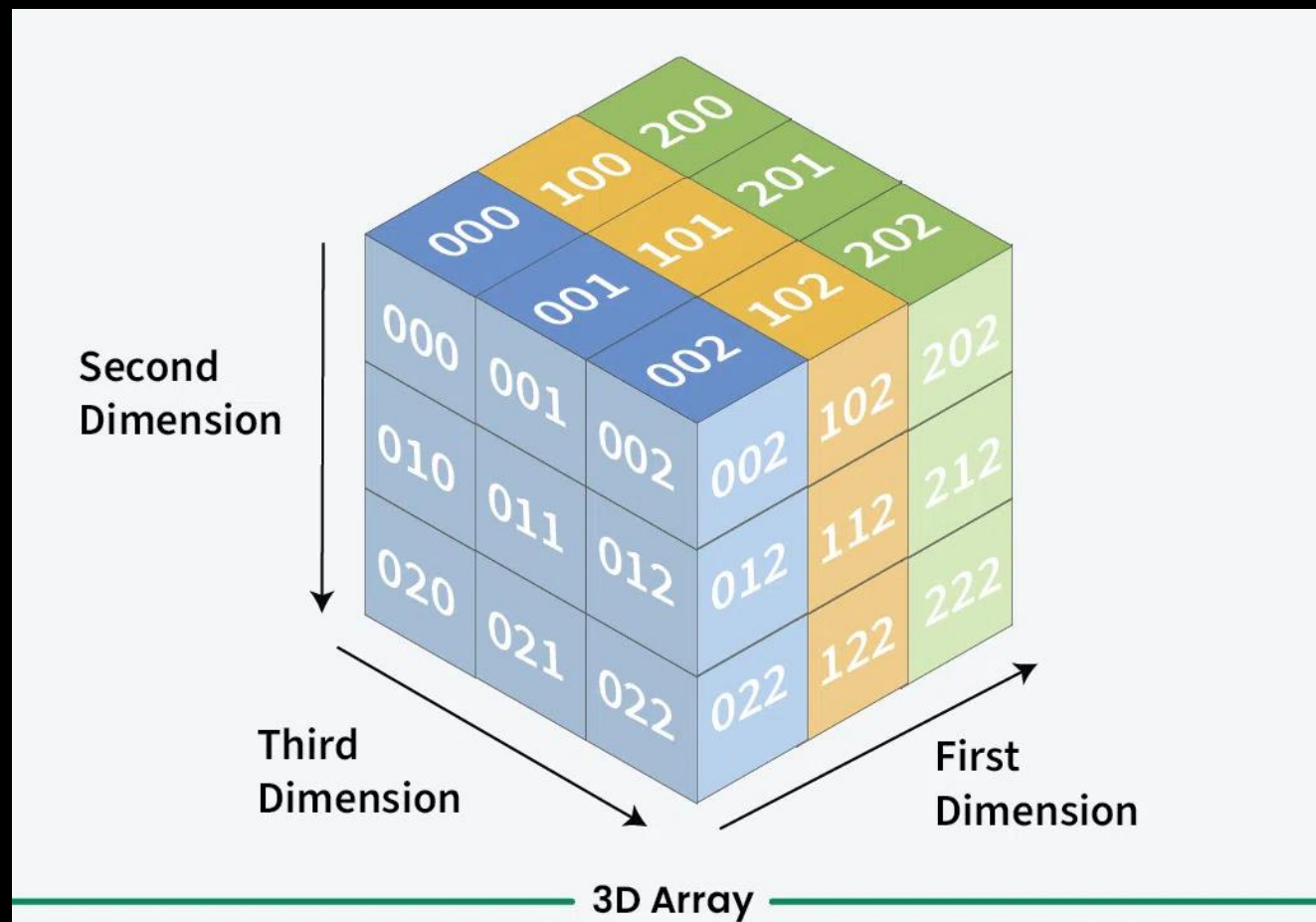


1	2	3
4	5	6
7	8	9

Main Diagonal

Secondary Diagonal

```
for (int i = 0; i < N; i++) {  
    for (int j = 0; j < N; j++) {  
        if (i == j) {  
            printf("%d ", matrix[i][j]);  
        }  
    }  
}
```



3D массив - это уже будет квартира с комнатами со шкафами с коробками.. 4D – дом с квартирами.. и так продолжать можно долго

Транспонирование

```
// квадратная матрица N × N, на месте
for (int i = 0; i < N; i++) {
    for (int j = i + 1; j < N; j++) { // j = i+1!
        int tmp = m[i][j];
        m[i][j] = m[j][i];
        m[j][i] = tmp;
    }
}
```

Свёртка: размытие 3×3

```
for (int i = 0; i < h; i++) {  
    for (int j = 0; j < w; j++) {  
        int sum = 0, cnt = 0;  
        for (int di = -1; di <= 1; di++) {  
            for (int dj = -1; dj <= 1; dj++) {  
                int ni = i + di, nj = j + dj;  
                if (ni >= 0 && ni < h && nj >= 0 && nj < w) {  
                    sum += src[ni][nj]; cnt++;  
                }  
            }  
        }  
        dst[i][j] = sum / cnt;  
    }  
}
```

У крайних пикселей нет 9 соседей.

Результат нужно писать в отдельную матрицу

Так работают фильтры и первые слои нейросетей (CNN)

```
char c1 = 'H';
```

```
char c2 = 'i';
```

```
char c3 = '!';
```

```
printf("%c %c %c\n", c1, c2, c3);
```

```
printf("%d %d %d\n", c1, c2, c3);
```

String

В языке Си строка - есть массив символов.

```
char s[] = "Hi!";  
printf("%s\n", s);
```

Формат в printf - "%s" позволяет выводить строки на экран (от слова string).

Как понять где строка закончится?

NUL

s[0] s[1] s[2] s[3]

	H	i	!	\0	
--	----------	----------	----------	-----------	--

s[0] s[1] s[2] s[3]

	72	105	33	0	
--	-----------	------------	-----------	----------	--

Нул-терминатор ('\0') - это символ конца строки

String array

```
char words[2][5] = {"Hi!", "Bye!"};
```

```
printf("%s\n", words[0]);
```

```
printf("%s\n", words[1]);
```

```
printf("%c%c%c\n", words[0][0], words[0][1],  
        words[0][2]);
```

```
printf("%c%c%c%c\n", words[1][0], words[1][1],  
        words[1][2], words[1][3]);
```

- **scanf()** - считывает до пробела, используем формат %s, не используем амперсанд, т.к. имя массива нам даст адрес переменной массива.
- **fgets()** - считывает с пробелами

```
char name[20];  
printf("Enter name: ");  
scanf("%s", name);  
printf("Your name is %s\n", name);
```

```
printf("Enter name: ");  
fgets(name, sizeof(name), stdin);  
printf("My name is %s\n", name);
```

stdin – стандартный ввод, то куда попадают наши вводимые данные в ОС

stdout – стандартный вывод

stderr – стандартный вывод для ошибок

```
char buffer[8];  
scanf("%s", buffer);
```

```
char buffer[8];  
scanf("%7s", buffer);
```

string.h

```
#include <string.h> // Для работы с функциями
                    // обработки строк

strlen();          // Длина строки, не включая '\0'
strcpy();           // Копирование одной строки в другую
strncpy();          // Тоже самое, но не выходя за размер n
puts();             // Вывод на экран строки
fgets();            // Ввод с клавиатуры строки
strcmp();           // Сравнение двух строк
strcat();           // Объединение двух строк
```

ctype.h и snprintf

```
#include <ctype.h>
```

```
isalpha(c); isdigit(c); isspace(c);  
isupper(c); toupper(c); tolower(c);
```

```
int count = 0;  
for (int i = 0; s[i] != '\0'; i++) {  
    if (isupper(s[i])) count++;  
}
```

```
#include <stdio.h>
```

```
char buf[64];  
snprintf(buf, sizeof(buf), "x = %d", 10); // безопасно
```

Sorting time

Сортировки - алгоритмы для упорядочивания данных в массивах (по возрастанию или убыванию). С отсортированными данными обычно проще и быстрее работать

Пару алгоритмов сортировок:

- Bubble Sort - Пузырьковая сортировка
- Insertion Sort - Сортировка вставками
- Selection Sort - Сортировка выбором
- Counting Sort - Сортировка подсчетом
- Radix Sort - Поразрядная сортировка
- Merge Sort - Сортировка слиянием
- Quick Sort - Быстрая сортировка

swap

```
int temp = array[0];  
array[0] = array[1];  
array[1] = temp;
```

*Имя temp или tmp - от слова временный (**temporary**)

Bubble sort

```
для i от 0 до n-1:  
    для j от 0 до n-i-1:  
        ЕСЛИ arr[j] > arr[j+1]:  
            поменять местами arr[j] и arr[j+1]
```

6 5 3 1 8 7 2 4

Bubble sort

```
void bubble_sort(int size, int arr[size]) {  
    for (int i = 0; i < size; i++) {  
        for (int j = 0; j < size - i - 1; j++) {  
            if (arr[j] > arr[j + 1]) {  
                int tmp = arr[j];  
                arr[j] = arr[j + 1];  
                arr[j + 1] = tmp;  
            }  
        }  
    }  
}
```

*В функцию можно передать массив, но из функции нельзя его вернуть

Insertion Sort (вставками)

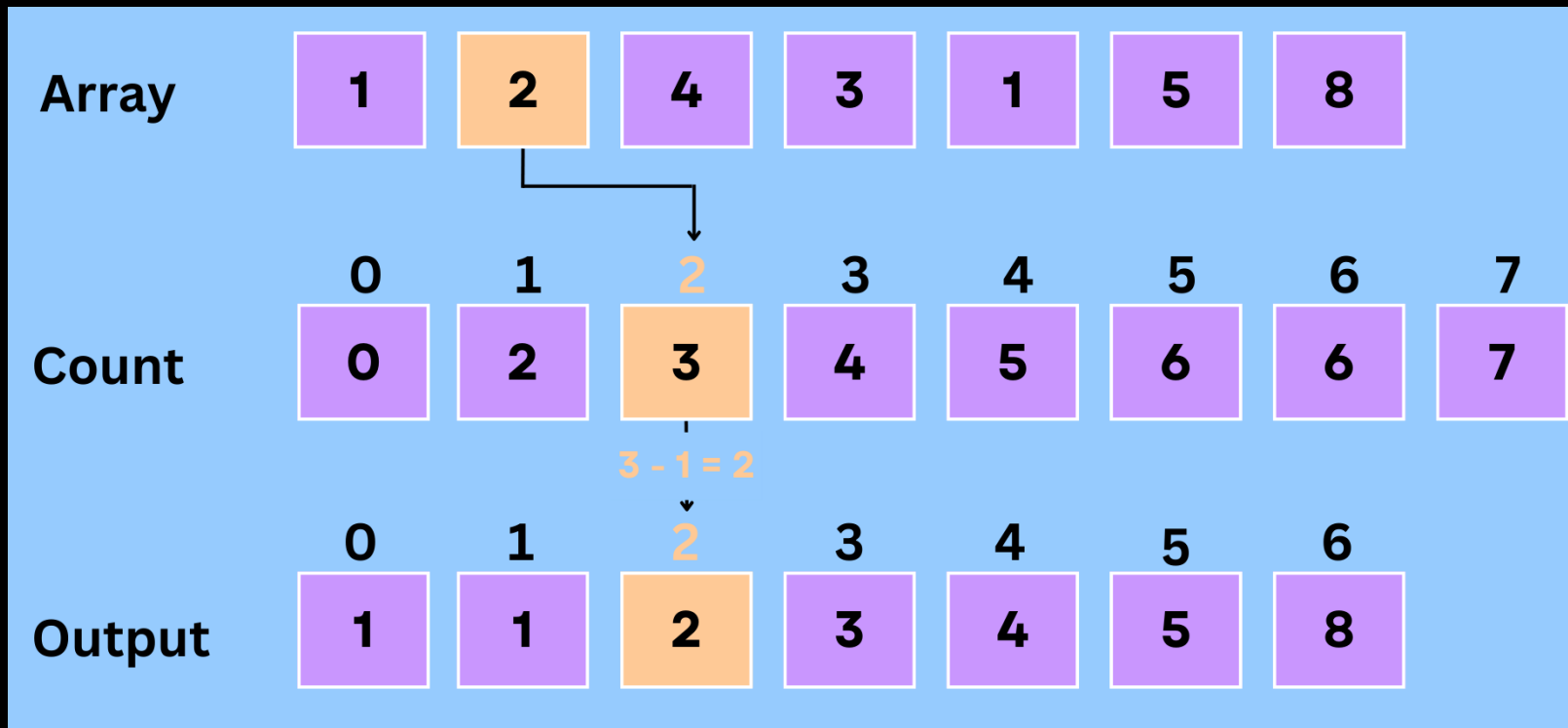
6 5 3 1 8 7 2 4

Selection sort (выбором)

	8
	5
	2
	6
	9
	3
	1
	4
	0
	7

Counting sort (Подсчетом)

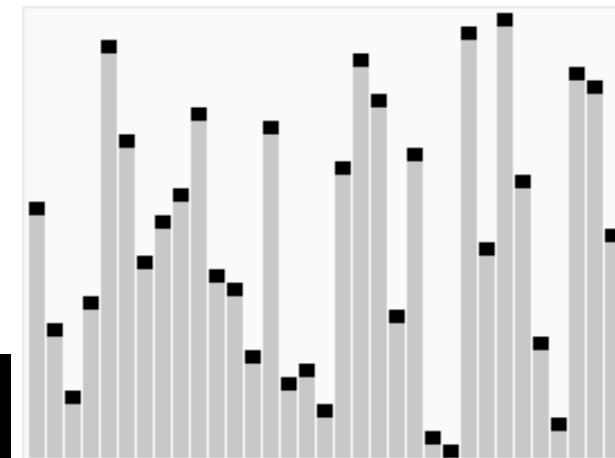
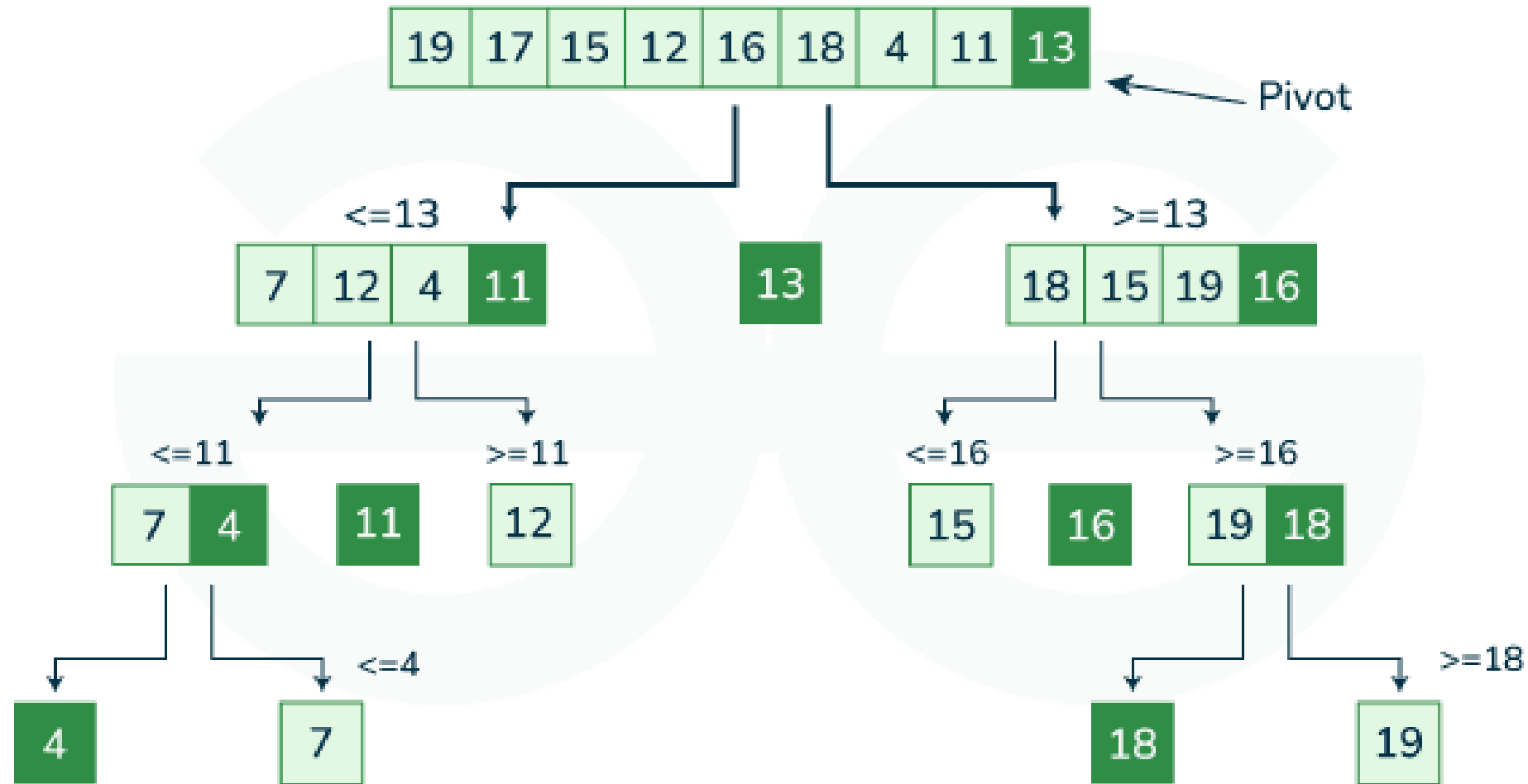
```
count[arr[i]]++;  
// i - индекс,  
// массив arr - оригинальный сортируемый массив,  
// count - массив для подсчета
```



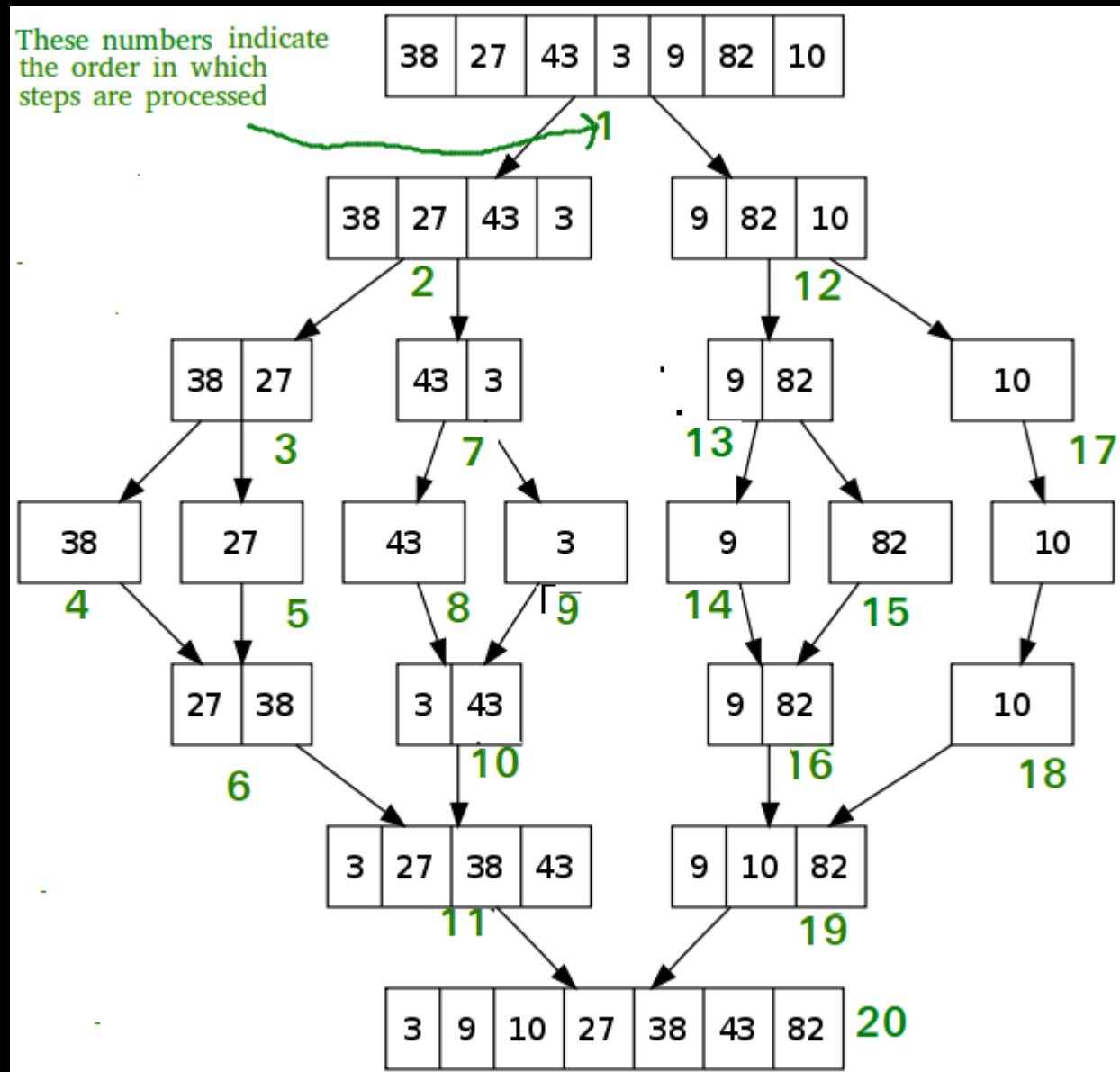
Radix sort (поразрядная)

Sort Digit 0	Sort Digit 1	Sort Digit 2	Final Result
9 5 4	4 1 1	0 0 9	0 0 9
3 5 4	9 5 4	4 1 1	3 5 4
0 0 9	3 5 4	9 5 4	4 1 1
4 1 1	0 0 9	3 5 4	9 5 4

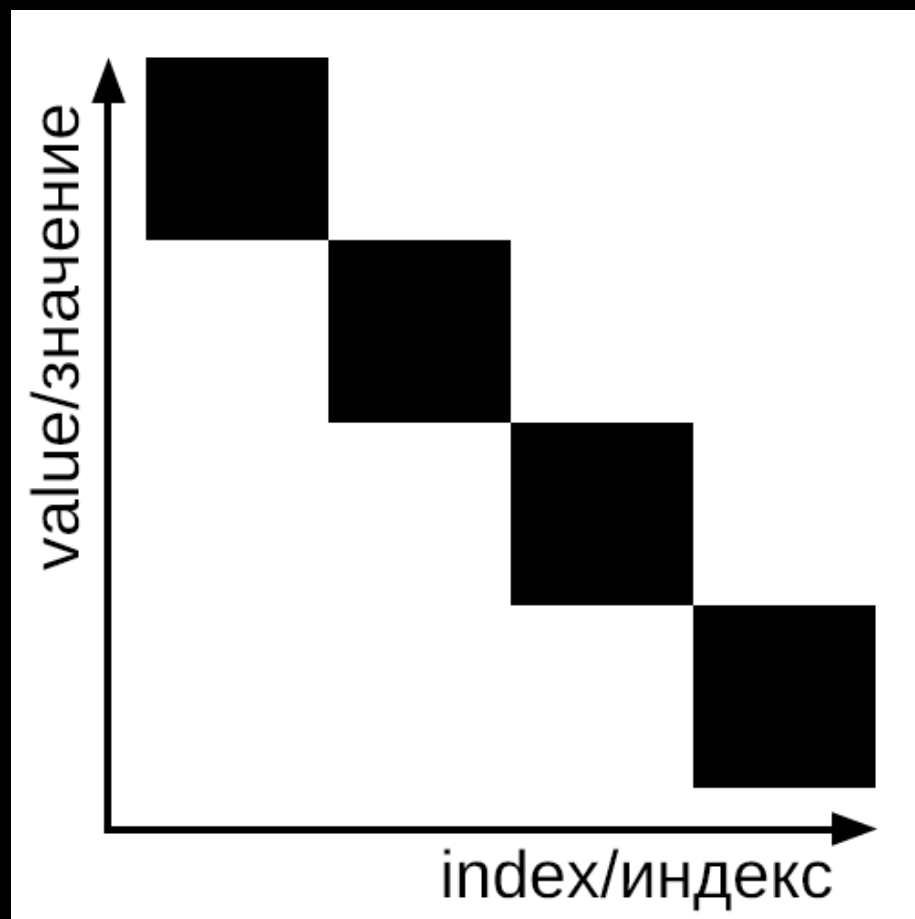
Quick sort (быстрая)



Merge sort (слиянием)



Bogo sort (непрактичная)



С такой сортировкой отсортировать массив любого размера можно с 1 раза, однако шанс не велик.. **но он есть**

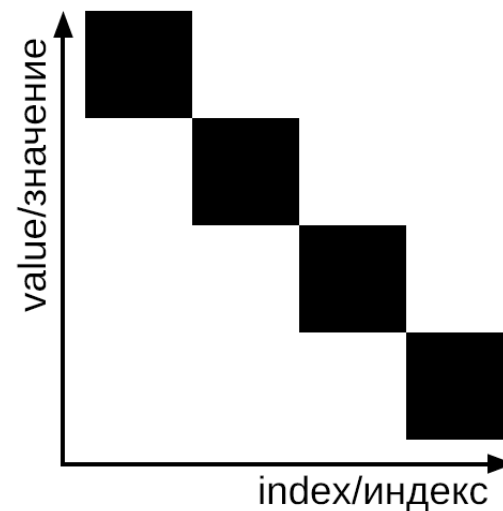
Bogo sort (непрактичная)

При прохождении цикла один раз в секунду сортировка в среднем может занять:

Кол-во элементов	Среднее время
1	1 с
2	4 с
3	18 с
4	96 с
5	10 мин
6	1,2 ч
7	9,8 ч
8	3,7 сут
9	37,8 сут
10	1,15 лет
11	13,9 лет
12	182 года

При работе 4-ядерного процессора на частоте 2,4 ГГц (9,6 млрд операций в секунду):

Кол-во элементов	Среднее время
10	0,0037 с
11	0,045 с
12	0,59 с
13	8,4 с
14	2,1 мин
15	33,6 мин
16	9,7 ч
17	7,29 сут
18	139 сут
19	7,6 лет
20	160 лет



Таким образом, колода в 32 карты будет сортироваться этим компьютером в среднем $2,7 \cdot 10^{19}$ лет.

Другие сортировки

Thanos sort - удаляем половину массива

Sleep sort - число секунд сна, потом печать

Miracle sort - ждём, вдруг отсортируется

Quantum bogosort - все перестановки во всех вселенных

...

Сложность алгоритмов (Big O)

Сложность - это про ресурсы:

время и память (time and space complexity).

$O(f(n))$ - как растёт работа при росте данных, а не количество секунд.

По умолчанию - худший случай.

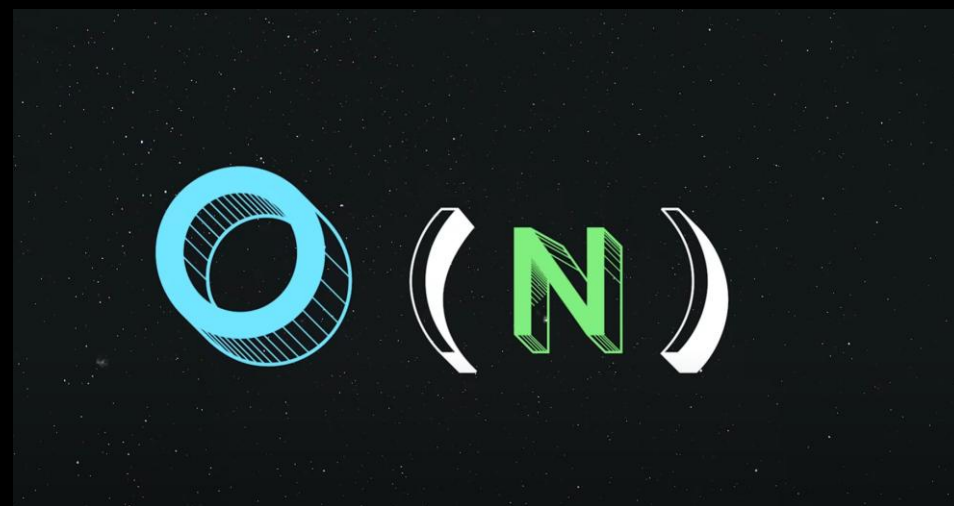
Константы и множители отбрасываем:

$$O(2n) = O(n), \quad O(3n) = O(n)$$

$$O(n^2 + n) = O(n^2)$$

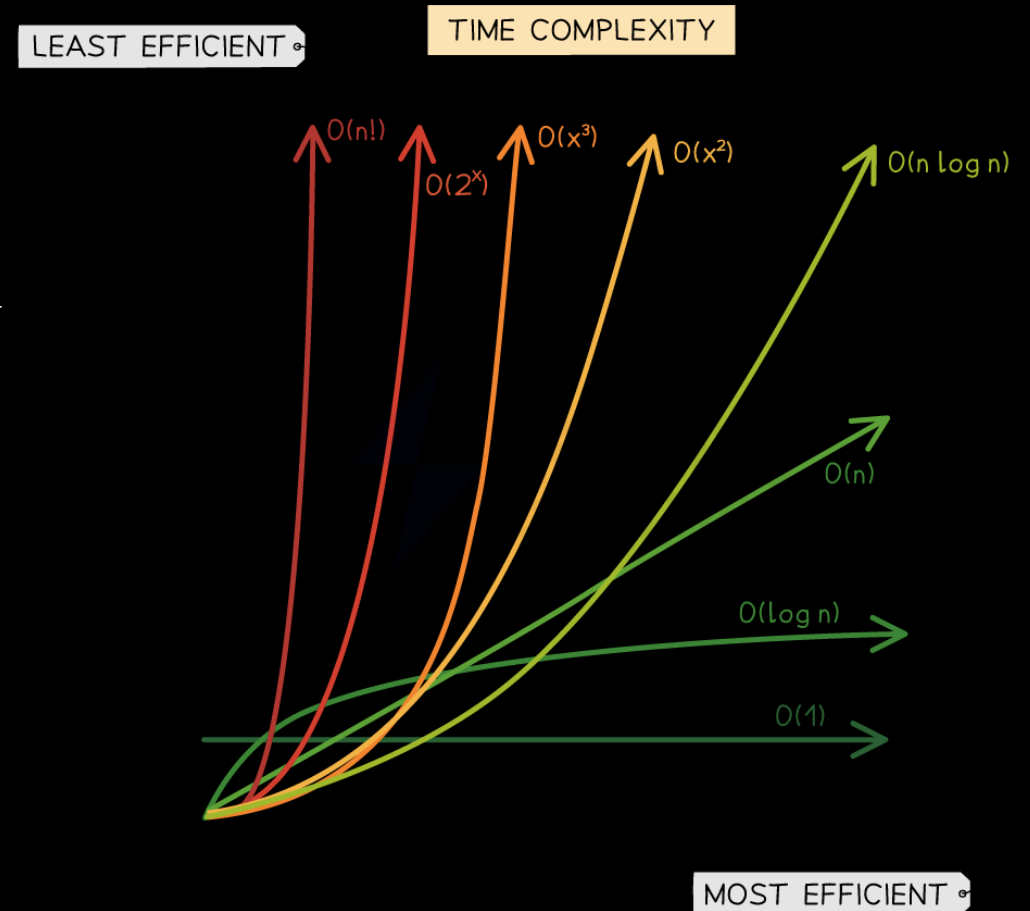
$$O(n^2) + O(\log n) = O(n^2)$$

$$O(n! + 999) = O(n!)$$



Классы сложности

- $O(1)$ - `arr[i]`
- $O(\log n)$ - бинарный поиск
- $O(n)$ - один проход по массиву
- $O(n \log n)$ - merge и quick sort
- $O(n^2)$ - вложенные циклы
- $O(2^n)$ - Фибоначчи рекурсией
- $O(n!)$ - перестановки



- 1) два вложенных цикла по n = ?
- 2) один проход по массиву = ?
- 3) $keys[n]$ из массива = ?
- 4) $for (i = n; i > 1; i /= 2)$ = ?

Оцени сложность сам

- 1) два вложенных цикла по n $= O(n^2)$
- 2) один проход по массиву $= O(n)$
- 3) `keys[n]` из массива $= O(1)$
- 4) `for (i = n; i > 1; i /= 2)` $= O(\log n)$

Окно и префиксные суммы

Кофейня: какие 3 дня подряд дали больше всего выручки?

В лоб - складывать 3 числа на каждую позицию.

Сдвинули окно: ушёл старый день, пришёл новый.

Префиксные суммы - накопительный итог, как в чеке.

Дни 2..4: $\text{prefix}[4] - \text{prefix}[1] = 6400 - 1500 = 4900$

5_arrays/src/array_advanced.c

Github

<https://github.com/kruffka/C-Programming>

